# Lecture 2

## SENG8323: SOFTWARE MODELING DESIGN

**LECTURER: Mr. Placide RWEGO MUCYO**

**CONTACTS: Phone number: 0788496945**

**E-mail: mupla2@gmail.com**

**Academic Year 2025-2026**

# Requirements Engineering

# Software development activities

- Software development always includes the following activities:
  - Requirements analysis or software requirements ( more concerned in this class)
  - Design
  - Construction
  - Testing (sometimes)
  - Maintenance
- These activities do not follow strictly one after another!
  - Often overlap and interact

# Software Requirements

- It is always hard to describe and document the requirements in comprehensive and not ambiguous way
  - Good requirements save time and money
- Requirements always change during the project!
  - Good software requirements specification reduces the changes
  - Prototypes significantly reduce changes

# What is requirement?

- A requirement is a statement about an intended product that specifies what it should do or how it should perform.
- *The requirements for a system are the descriptions of what the system should do—*

*the services that it provides and the constraints on its operation.*

- Goal: To make as specific, unambigous, and clear as possible.

# Requirement engineering definition

- The process of defining, documenting, and maintaining requirements in the engineering design process provides the appropriate mechanism to understand what the customer desires, analyzing the need, and assessing feasibility, negotiating a reasonable solution, specifying the solution clearly, validating the specifications and managing the requirements as they are transformed into a working system

# Requirements Gathering Techniques

- Questionnaires: Series of questions designed to elicit specific information from us. The questions may require different kinds of answers: some require a simple Yes/No, others ask us to choose from a set of pre-supplied answers.
- Interviews: Interviews involve asking someone a set of questions. Often interviews are face-to-face, but they don't have to be

# Requirements Gathering Techniques (continued)

- Interviews:
  - Forum for talking to people
  - Structured, unstructured or semi-structured
  - Good for exploring issues
  - But are time consuming and may be infeasible to visit everyone

# Requirement gathering techniques

- Focus groups and workshops: Interviews tend to be one on one, and elicit only one person's perspective. It can be very revealing to get a group of stakeholders together to discuss issues and requirements.

- **Naturalistic Observation**: It can be very difficult for humans to explain what they do or to even describe accurately how they achieve a task.

# Requirement gathering Techniques (continued)

- Naturalistic observation:
  - Spend time with stakeholders in their day-to-day tasks, observing work as it happens
  - Gain insights into stakeholders' tasks
  - Good for understanding the nature and context of the tasks
  - But, it requires time and commitment from a member of the design team, and it can result in a huge amount of data

# Requirements gathering (continued)

- Studying documentation: Procedures and rules are often written down in a manual and these are a good source of data about the steps involved in an activity and any regulations governing a task.

**Use All of the above In Combination : Constraints of Time and Money**

# Overview of requirement gathering methods

| Technique | Good for | Kind of data | Advantages | Disadvantages |
|---|---|---|---|---|
| **Questionnaires** | Answering specific questions | Quantitative and qualitative data | Can reach many people with low resource | The design is crucial. Response rate may be low. Responses may not be what you want |
| **Interviews** | Exploring issues | Some quantitative but mostly qualitative data | Interviewer can guide interviewee if necessary. Encourages contact between developers and users | Time consuming. Artificial environment may intimidate interviewee |
| **Focus groups and workshops** | Collecting multiple viewpoints | Some quantitative but mostly qualitative data | Highlights areas of consensus and conflict. Encourages contact between developers and users | Possibility of dominant characters |
| **Naturalistic observation** | Understanding context of user activity | Qualitative | Observing actual work gives insights that other techniques can't give | Very time consuming. Huge amounts of data |
| **Studying documentation** | Learning about procedures, regulations and standards | Quantitative | No time commitment from users required | Day-to-day working will differ from documented procedures |

# Requirements Gathering Guidelines

- Focus on identifying the stakeholders
- Involve all the stakeholder groups
- Need more than on person from stakeholder group(s)
- **Use a combination of data gathering techniques**
  For example: use observation to understand the context, interviews to target specific user groups, questionnaires to reach a wider population, and **focus groups to build a consensus view**

# Requirement Gathering Guidelines Cont.

- Run a pilot session if possible to ensure that your data-gathering session is likely to go as planned
- In an ideal world, you would understand what you are looking for and what kinds of analysis you want to do, and design the data-capture exercise to collect the data you want.
- **However, requirement gathering is expensive and often a tightly constrained resource.**

# Understanding Requirements

- Understanding the requirements of a problem is among the most difficult tasks that face a software engineer.
- When you first think about it, developing a clear understanding of requirements doesn't seem that hard.
- Two questions:
- Doesn't the customer know what is required?
- Shouldn't the end users have a good understanding of the features and functions that will provide benefit?
- Surprisingly, in many instances the answer to these questions is No.

# Understanding Requirements- What is it?

- Before you begin any technical work, it's a good idea to apply a set of requirements engineering tasks.
- These tasks lead to an understanding of what the business impact of the software will be, what the customer wants, and how end users will interact with the software

- Even if customers and end-users are explicit in their needs, those needs will change throughout the project

# Understanding Requirements- Who does it?

- Software engineers (system engineers or "analysts" in the IT world) and other project stakeholders (managers, customers, end users) all participate in requirements engineering.

# Understanding Requirements- Why is it important?

- Designing and building an elegant computer program that solves the wrong problem serves no one's needs.
- That's why it's important to understand what the customer wants before you begin to design and build a computer-based system

# Requirements engineering

- The broad spectrum of tasks and techniques that lead to an understanding of requirements is called *requirements engineering.*
- The process of establishing the services that the customer requires from a system and the constraints under which it operates and is developed
- The requirements themselves are the descriptions of the system services and constraints that are generated during the requirements engineering process

# Types of requirements

- User requirements
  - Statements in natural language plus diagrams of the services the system provides and its operational constraints. Written for customers
- System requirements
  - A structured document setting out detailed descriptions of the system services. Written as a contract between client and contractor
  - Note: There are user/system functional/non-functional requirements.

# Types of requirements

- Functional requirements
  - Services the system should provide
  - What the system should do or not in reaction to particular situations
- Non-functional requirements
  - Constraints on the services or functions offered by the system
  - Examples: Timing constraints, constraints on the development process

# Functional requirements

- Describe functionality or system services
- Depend on the type of software, expected users and the type of system where the software is used
- Functional user requirements may be high-level statements of what the system should do but functional system requirements should describe the system services in detail

# Examples of functional requirements

- For ATM System
- The system shall enable the customers of ABC Bank, who have valid ATM cards, to perform three types of transactions;

  1) withdrawal of funds,

  2) Query of account balance, and

3) transfer of funds from one bank account to another account in the same bank

- The system shall confiscate the ATM card if it detects that a lo or stolen card has been inserted by a customer.

# Non-functional requirements

- Define system properties and constraints e.g. reliability, response time and storage requirements.
- Constraints are I/O device capability, system representations, etc.
- Non-functional requirements may be more critical than functional requirements.
- If these are not met, the system is useless

- For example for ATM machine, the system can be available 24hrs a day

# Non-functional classifications

- Product requirements
  - Requirements which specify that the delivered product must behave in a particular way e.g. execution speed, reliability, etc.
- Organisational requirements
  - Requirements which are a consequence of organisational policies and procedures e.g. process standards used, implementation requirements, etc.
- External requirements
  - Requirements which arise from factors which are external to the system and its development process e.g. interoperability requirements, legislative requirements, etc.

# Non-functional requirements cont..

- It is important to be able to test/verify/check non-functional requirements

| Property | Measure |
|---|---|
| Speed | Processed transactions/second<br>User/Event response time<br>Screen refresh time |
| Size | K Bytes<br>Number of RAM chips |
| Ease of use | Training time<br>Number of help frames |
| Reliability | Mean time to failure<br>Probability of unavailability<br>Rate of failure occurrence<br>Availability |
| Robustness | Time to restart after failure<br>Percentage of events causing failure<br>Probability of data corruption on failure |
| Portability | Percentage of target dependent statements<br>Number of target systems |

# User requirements

- Should describe functional and non-functional requirements so that they are understandable by system users who don't have detailed technical knowledge
- User requirements are defined using natural language, tables and diagrams
- Problems:

– Lack of clarity - ambiguous language
– Requirements confusion - functional, non-functional requirements, design information are not distinguished
– Incompleteness – requirements may be missing
– Inconsistency – requirements may contradict themselves

# Problems with natural language

- Lack of clarity
  - Precision is difficult without making the document difficult to read
- Requirements confusion
  - Functional and non-functional requirements tend to be mixed-up
- Requirements amalgamation
  - Several different requirements may be expressed together

# User requirements

- Guideline to minimize the issues:
  - separate functional and non-functional requirements,
  - requirements must be clearly identified

# System requirements

- A structured document setting out detailed descriptions of the system services.
- Written as a contract between client and contractor.
- Elaborate the user requirements to get a precise, detailed and complete version of them
- Used by designers and developers
- An important aspect is how to present/write system requirements
  - Natural language is often used!

# Requirements Example

- **User requirement**: The library system should provide a way to allow a student to borrow a book from the library.
- **System requirement**: The library system should provide a withdraw interaction that allows a student to withdraw a book given the *isbn* and copy *number* of the book to be withdrawn.
- The interaction fails if: the book is already withdrawn, the book is not in the library's collection, the student has already withdrawn 5 books, the book is on hold by someone else.

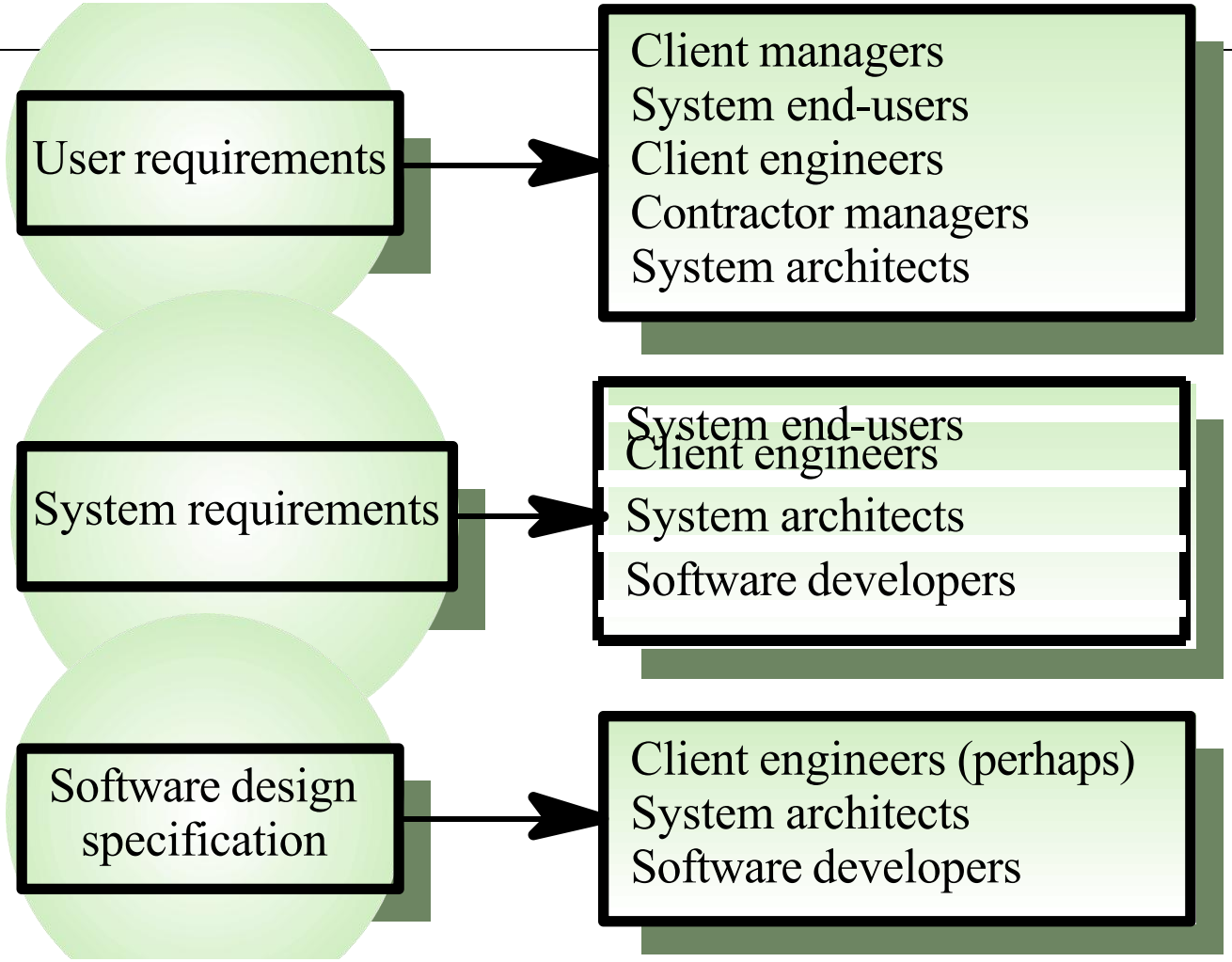
# Why requirements?

- What are the advantages of a complete set of documented requirements?
  - Ensures the user (not the developer) drives system functionalities
  - Helps avoiding confusion and arguments
  - Helps minimizing the changes
- Changes in requirements are expensive. Changing the requirements costs:
  - 3 x as much during the design phase
  - 5-10 x as much during implementation
  - 10-100 x as much after release

# Why requirements?

- 2/3 of finished system errors are requirements and design errors
- A careful requirements process doesn't mean there will be no changes later
  - Average project experiences about 25% changes in the requirements
  - This accounts for 70-80% if the rework of the project
  - Important to plan for requirements changes

# Requirements readers



# Requirements Engineering Activities

| | |
|---|---|
| ***Feasibility study*** | Determine if the *user needs* can be *satisfied* with the *available technology* and *budget*. |
| ***Requirements analysis*** | Find out *what system stakeholders require* from the system. |
| ***Requirements definition*** | *Define the requirements* in a form understandable to the customer. |
| ***Requirements specification*** | Define the requirements in *detail*. (Written as a contract between client and contractor.) |

*"Requirements are for users; specifications are for analysts and developers."*

# Feasibility studies

- A feasibility study decides whether or not the proposed system is worthwhile
- A short focused study that checks
  - If the system can be engineered using current technology and within budget
  - If the system contributes to organisational objectives
  - If the system can be integrated with other systems that are used

# Feasibility study implementation

- Based on information assessment (what is required), information collection and report writing
- Questions for people in the organisation
  - ❑ What if the system wasn't implemented?
  - ❑ How will the proposed project help?
  - ❑ How does the proposed project contribute to the overall objectives of the organization?
  - ❑ Do technical resources exist?
  - ❑ What will be the integration problems?
  - ❑ Is new technology needed? What skills?
  - ❑ What is the risk associated with the technology?
  - ❑ What facilities must be supported by the proposed system?

# Requirements Analysis

Sometimes called *requirements elicitation* or *requirements discovery*

Technical staff work with customers to determine

- the application *domain*,
- the *services* that the system should provide and
- the system's operational *constraints*.

Involves various *stakeholders*:

- e.g., end-users, managers, engineers involved in maintenance, domain experts, trade unions, etc.

# Writing system requirements

- Natural language (informal requirements)
  - Reviled by academics
  - But widely practiced: everybody can read them, finding a better notation is hard
- Structured natural language
  - Forms/templates are used to bring some rigor to natural language presentations
- Graphical notations
  - Using boxes, arrows… but they mean different things to different people
- Formal specification
  - Based on logic, state machines…
  - Hard to understand for many people

# Problems of requirements analysis

- Stakeholders don’t know what they really want
- Stakeholders express requirements in their own terms
- Different stakeholders may have conflicting requirements
- Organisational and political factors may influence the system requirements
- The requirements change during the analysis process. New stakeholders may appear and the business environment change

# Assignment 2

- Select a software project topic of your choice that solve a particular problem
- Use any requirement gathering method
- Write your project requirements ( at least 2 page)